statistical_analysis_horses.R

john.little

2026-10-16

```
## install.packages(c("dplyr", "stringr", "car", "ggplot2"))
# ALL RESULTS ARE BASED ON A VERY SMALL SAMPLE SIZE - RELOAD WITH FULL SAMPLE BEFORE CONCLUDING ANYTHIN

# load dataset
data <- read.csv("C:/Users/john.little/OneDrive - Churchill Downs Incorporated/Documents/testing.horses


# ** FILTERS & SET UP ** ---------------------------------------------------------
# load libraries
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(car)
```

```
## Loading required package: carData

##
## Attaching package: 'car'

## The following object is masked from 'package:dplyr':
##
##     recode
```

```
library(survival)

# model components used in every analysis
core_components <- c("C_SPEED","C_POWER","C_CLASS","C_DRIVER",
                     "C_TRAINER","C_EARLY","C_FINISH","C_RECENCY","C_MARKET")
```

```r
# set filters (comma-separated, case-insensitive; "" = all values in the dataset)
filter_track   <- ""
filter_breed   <- ""
filter_weights <- ""
filter_version <- "v2.2, v2.3"

# filter function
apply_filter <- function(df, col, txt) {
  if (is.null(txt) || trimws(txt) == "") return(df)
  keep <- trimws(strsplit(txt, ",")[[1]])
  missing_vals <- keep[!tolower(keep) %in% tolower(df[[col]])]
  if (length(missing_vals) > 0) warning(col, ": not found in data: ", paste(missing_vals, collapse = ",
  df[tolower(df[[col]]) %in% tolower(keep), ]
}

# apply filters
data <- data |>
  apply_filter("TRACK", filter_track) |>
  apply_filter("BREED", filter_breed) |>
  apply_filter("WEIGHT_SET", filter_weights) |>
  apply_filter("MODEL_VERSION", filter_version)

# check what's left
stopifnot(nrow(data) > 0)
n_distinct(data$RACE_ID)
```

```
## [1] 25
```

```r
nrow(data)
```

```
## [1] 188
```

```r
# decision function
decide <- function(p, alpha = 0.05) {
  cat("p-value =", signif(p, 4), "->", ifelse(p < alpha, "reject H0", "fail to reject H0"), "\n")
}

# cohen's d function
cohens_d <- function(x, g) {
  x1 <- x[g == 1]
  x0 <- x[g == 0]
  (mean(x1) - mean(x0)) / sqrt(((length(x1) - 1) * var(x1) + (length(x0) - 1) * var(x0)) /
                                 (length(x1) + length(x0) - 2))
}


# ** SUMMARY OF DATA ** ----------------------------------------------------
# summarize and understand data
head(data)
```

```
##                         RACE_ID WINNER  TOP3  DATE POST_TIME SNAPSHOT_TIME
```

```
## 1 2026-10-02_Gulfstream Park_3      7 7,2,3 46297    1:25pm     0.6058796
## 2 2026-10-02_Gulfstream Park_3      7 7,2,3 46297    1:25pm     0.6058796
## 3 2026-10-02_Gulfstream Park_3      7 7,2,3 46297    1:25pm     0.6058796
## 4 2026-10-02_Gulfstream Park_3      7 7,2,3 46297    1:25pm     0.6058796
## 5 2026-10-02_Gulfstream Park_3      7 7,2,3 46297    1:25pm     0.6058796
## 6 2026-10-02_Gulfstream Park_3      7 7,2,3 46297    1:25pm     0.6058796
##   MIN_TO_POST           TRACK RACE_NUM        BREED MODEL_VERSION   WEIGHT_SET
## 1           0 Gulfstream Park        3 Thoroughbred          v2.2 Thoroughbred
## 2           0 Gulfstream Park        3 Thoroughbred          v2.2 Thoroughbred
## 3           0 Gulfstream Park        3 Thoroughbred          v2.2 Thoroughbred
## 4           0 Gulfstream Park        3 Thoroughbred          v2.2 Thoroughbred
## 5           0 Gulfstream Park        3 Thoroughbred          v2.2 Thoroughbred
## 6           0 Gulfstream Park        3 Thoroughbred          v2.2 Thoroughbred
##   FIELD_SIZE                              TRACK_CONDITION WETNESS_TIER DISTANCE
## 1          7 Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast            0      550
## 2          7 Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast            0      550
## 3          7 Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast            0      550
## 4          7 Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast            0      550
## 5          7 Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast            0      550
## 6          7 Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast            0      550
##   DIST_UNIT DIST_TEXT SURFACE RACE_TYPE RACE_CLASS PURSE CLAIM_PRICE SPEED_PAR
## 1     FALSE   5 1/2 F    Dirt       MCL   MD 12500 25000       12500        72
## 2     FALSE   5 1/2 F    Dirt       MCL   MD 12500 25000       12500        72
## 3     FALSE   5 1/2 F    Dirt       MCL   MD 12500 25000       12500        72
## 4     FALSE   5 1/2 F    Dirt       MCL   MD 12500 25000       12500        72
## 5     FALSE   5 1/2 F    Dirt       MCL   MD 12500 25000       12500        72
## 6     FALSE   5 1/2 F    Dirt       MCL   MD 12500 25000       12500        72
##   WEEK_RACES MEET_RACES PROGRAM        HORSE_NAME POST RUN_STYLE RUN_STYLE_PTS
## 1          2         29       2        Senta Says    2       E/P             5
## 2          2         29       4   Diamond's Honor    4         E             6
## 3          2         29       8      Camm's Girl    8         S             0
## 4          2         29       7  Starship Daytona    7      <NA>             4
## 5          2         29       3     Cousin Pammie    3      <NA>             0
## 6          2         29       1 Memory of My Life    1      <NA>             0
##   ODDS ODDS_SRC C_SPEED C_POWER C_CLASS C_DRIVER C_TRAINER C_EARLY C_FINISH
## 1  1.2     LIVE    67.0   110.2   106.8    18.68     22.90    91.5       66
## 2  5.0     LIVE    64.0   105.6   106.8    11.38     16.00    91.0       63
## 3  7.0     LIVE    69.0    99.3   105.2    16.12     10.00    82.0       80
## 4  1.5     LIVE    58.6   101.3   107.4    20.11     14.29    86.5       63
## 5 25.0     LIVE    54.0    97.4   103.4    16.56      3.45    81.5       61
## 6 52.0     LIVE    48.6    87.7   101.8     6.12      2.78    81.5       55
##   C_RECENCY C_MARKET POST_BONUS STYLE_BONUS WET_BONUS FINAL_SCORE RANK Z_SCORE
## 1       100   45.455       2.16        0.00         0       83.53    1  1.6923
## 2       100   16.667       0.00        2.97         0       80.81    2  1.1119
## 3        70   12.500       0.00        0.00         0       76.77    3  0.2499
## 4        50   40.000       0.00        0.00         0       74.50    4 -0.2344
## 5       100    3.846       2.16        0.00         0       72.40    5 -0.6825
## 6       100    1.887       5.00        0.00         0       71.19    6 -0.9406
##   WIN_PROB IMPUTED_COUNT Is.Winner In.Top3
## 1   0.3282             0         0       1
## 2   0.2258             0         0       0
## 3   0.1332             1         0       0
## 4   0.1011             2         1       1
## 5   0.0796             0         0       1
```

```
## 6   0.0700             0         0       0
```

```
summary(data)
```

```
##       RACE_ID        WINNER             TOP3          DATE
##  Length   :188   Min.   : 1.000   Length   :188   Min.   :46297
##  N.unique : 25   1st Qu.: 2.000   N.unique : 24   1st Qu.:46300
##  N.blank  :  0   Median : 4.000   N.blank  :  0   Median :46310
##  Min.nchar: 20   Mean   : 4.505   Min.nchar:  5   Mean   :46306
##  Max.nchar: 35   3rd Qu.: 7.000   Max.nchar:  6   3rd Qu.:46310
##                  Max.   :11.000                   Max.   :46310
##
##      POST_TIME   SNAPSHOT_TIME     MIN_TO_POST       TRACK        RACE_NUM
##  Length   :188   Min.   :0.6059   Min.   :0    Length   :188   Min.   :1.000
##  N.unique : 19   1st Qu.:0.6708   1st Qu.:0    N.unique :  6   1st Qu.:2.000
##  N.blank  :  0   Median :0.6806   Median :0    N.blank  :  0   Median :2.500
##  Min.nchar:  6   Mean   :0.6704   Mean   :0    Min.nchar:  7   Mean   :2.745
##  Max.nchar:  8   3rd Qu.:0.6882   3rd Qu.:0    Max.nchar: 22   3rd Qu.:4.000
##                  Max.   :0.6930   Max.   :0                    Max.   :6.000
##                                   NAs    :40
##        BREED       MODEL_VERSION     WEIGHT_SET    FIELD_SIZE
##  Length   :188   Length   :188   Length   :188   Min.   : 6.000
##  N.unique :  1   N.unique :  2   N.unique :  2   1st Qu.: 7.000
##  N.blank  :  0   N.blank  :  0   N.blank  :  0   Median : 7.000
##  Min.nchar: 12   Min.nchar:  4   Min.nchar:  7   Mean   : 7.819
##  Max.nchar: 12   Max.nchar:  4   Max.nchar: 12   3rd Qu.: 9.000
##                                                  Max.   :12.000
##
##   TRACK_CONDITION  WETNESS_TIER    DISTANCE      DIST_UNIT
##  Length   :188    Min.   :0     Min.   : 550.0   Mode :logical
##  N.unique :  7    1st Qu.:0     1st Qu.: 600.0   FALSE:175
##  N.blank  : 40    Median :0     Median : 650.0   NAs  :13
##  Min.nchar:  0    Mean   :0     Mean   : 680.3
##  Max.nchar: 44    3rd Qu.:0     3rd Qu.: 800.0
##                   Max.   :0     Max.   :1000.0
##                                 NAs    :13
##      DIST_TEXT        SURFACE        RACE_TYPE       RACE_CLASS
##  Length   :188   Length   :188   Length   :188   Length   :188
##  N.unique :  8   N.unique :  3   N.unique :  9   N.unique : 21
##  N.blank  : 13   N.blank  : 59   N.blank  : 13   N.blank  : 13
##  Min.nchar:  0   Min.nchar:  0   Min.nchar:  0   Min.nchar:  0
##  Max.nchar:  8   Max.nchar:  4   Max.nchar:  3   Max.nchar: 13
##
##
##      PURSE         CLAIM_PRICE      SPEED_PAR       WEEK_RACES
##  Min.   : 12200   Min.   :    0   Min.   : 0.00   Min.   : 0.000
##  1st Qu.: 33900   1st Qu.:    0   1st Qu.:72.00   1st Qu.: 2.000
##  Median : 41300   Median :15000   Median :79.00   Median : 6.000
##  Mean   : 54086   Mean   :22663   Mean   :75.61   Mean   : 7.737
##  3rd Qu.: 65000   3rd Qu.:30000   3rd Qu.:87.00   3rd Qu.:10.000
##  Max.   :175000   Max.   :80000   Max.   :92.00   Max.   :23.000
##  NAs    :13       NAs    :13      NAs    :13      NAs    :13
##    MEET_RACES       PROGRAM          HORSE_NAME        POST
##  Min.   :  1.0   Min.   : 1.000   Length   :188   Min.   : 1.000
```

```
##  1st Qu.: 10.0   1st Qu.: 3.000   N.unique :187   1st Qu.: 3.000
##  Median : 17.0   Median : 5.000   N.blank  :  0   Median : 5.000
##  Mean   : 79.6   Mean   : 5.005   Min.nchar:  5   Mean   : 5.005
##  3rd Qu.:125.0   3rd Qu.: 7.000   Max.nchar: 18   3rd Qu.: 7.000
##  Max.   :343.0   Max.   :15.000                   Max.   :15.000
##  NAs    :13
##      RUN_STYLE   RUN_STYLE_PTS        ODDS            ODDS_SRC
##  Length   :188   Min.   :0.000   Min.   : 0.11   Length   :188
##  N.unique :  4   1st Qu.:0.000   1st Qu.: 3.50   N.unique :  1
##  N.blank  :  0   Median :3.000   Median : 7.00   N.blank  :  0
##  Min.nchar:  1   Mean   :2.973   Mean   :12.62   Min.nchar:  4
##  Max.nchar:  3   3rd Qu.:5.000   3rd Qu.:15.00   Max.nchar:  4
##  NAs      : 44   Max.   :8.000   Max.   :83.00
##
##     C_SPEED         C_POWER         C_CLASS         C_DRIVER
##  Min.   :26.00   Min.   : 78.1   Min.   : 96.7   Min.   : 1.61
##  1st Qu.:61.70   1st Qu.:103.2   1st Qu.:106.8   1st Qu.:11.11
##  Median :69.83   Median :111.9   Median :109.1   Median :14.38
##  Mean   :67.97   Mean   :111.3   Mean   :108.9   Mean   :14.44
##  3rd Qu.:77.03   3rd Qu.:118.5   3rd Qu.:111.3   3rd Qu.:17.01
##  Max.   :96.00   Max.   :160.4   Max.   :117.6   Max.   :25.79
##
##    C_TRAINER        C_EARLY         C_FINISH        C_RECENCY
##  Min.   : 2.27   Min.   :50.50   Min.   : 24.00   Min.   : 50.00
##  1st Qu.:10.00   1st Qu.:76.88   1st Qu.: 64.75   1st Qu.: 85.00
##  Median :13.96   Median :82.00   Median : 74.40   Median :100.00
##  Mean   :14.06   Mean   :81.33   Mean   : 72.78   Mean   : 90.88
##  3rd Qu.:17.21   3rd Qu.:87.62   3rd Qu.: 81.00   3rd Qu.:100.00
##  Max.   :33.33   Max.   :99.00   Max.   :102.00   Max.   :100.00
##
##     C_MARKET       POST_BONUS      STYLE_BONUS       WET_BONUS  FINAL_SCORE
##  Min.   : 1.19   Min.   :0.0000   Min.   :0.0000   Min.   :0   Min.   :48.52
##  1st Qu.: 6.25   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0   1st Qu.:70.28
##  Median :12.50   Median :0.0000   Median :0.0000   Median :0   Median :74.55
##  Mean   :16.89   Mean   :0.3721   Mean   :0.4311   Mean   :0   Mean   :75.96
##  3rd Qu.:22.22   3rd Qu.:0.0600   3rd Qu.:0.0000   3rd Qu.:0   3rd Qu.:82.20
##  Max.   :90.00   Max.   :5.0000   Max.   :5.0000   Max.   :0   Max.   :99.36
##
##       RANK          Z_SCORE              WIN_PROB      IMPUTED_COUNT
##  Min.   : 1.00   Min.   :-2.2155000   Min.   :0.0272   Min.   : 0.000
##  1st Qu.: 2.00   1st Qu.:-0.6387750   1st Qu.:0.0717   1st Qu.: 0.000
##  Median : 4.00   Median :-0.0742500   Median :0.1087   Median : 0.500
##  Mean   : 4.41   Mean   : 0.0000016   Mean   :0.1330   Mean   : 2.218
##  3rd Qu.: 6.00   3rd Qu.: 0.6716500   3rd Qu.:0.1553   3rd Qu.: 2.000
##  Max.   :12.00   Max.   : 2.2271000   Max.   :0.4736   Max.   :13.000
##
##    Is.Winner        In.Top3
##  Min.   :0.000   Min.   :0.0000
##  1st Qu.:0.000   1st Qu.:0.0000
##  Median :0.000   Median :0.0000
##  Mean   :0.133   Mean   :0.3989
##  3rd Qu.:0.000   3rd Qu.:1.0000
##  Max.   :1.000   Max.   :1.0000
##
```

```r
str(data)
```

```
## 'data.frame':    188 obs. of  52 variables:
##  $ RACE_ID        : chr  "2026-10-02_Gulfstream Park_3" "2026-10-02_Gulfstream Park_3" "2026-10-02_Gu
##  $ WINNER         : int  7 7 7 7 7 7 7 5 5 5 ...
##  $ TOP3           : chr  "7,2,3" "7,2,3" "7,2,3" "7,2,3" ...
##  $ DATE           : int  46297 46297 46297 46297 46297 46297 46297 46297 46297 46297 ...
##  $ POST_TIME      : chr  "1:25pm" "1:25pm" "1:25pm" "1:25pm" ...
##  $ SNAPSHOT_TIME  : num  0.606 0.606 0.606 0.606 0.606 ...
##  $ MIN_TO_POST    : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ TRACK          : chr  "Gulfstream Park" "Gulfstream Park" "Gulfstream Park" "Gulfstream Park" ...
##  $ RACE_NUM       : int  3 3 3 3 3 3 3 1 1 1 ...
##  $ BREED          : chr  "Thoroughbred" "Thoroughbred" "Thoroughbred" "Thoroughbred" ...
##  $ MODEL_VERSION  : chr  "v2.2" "v2.2" "v2.2" "v2.2" ...
##  $ WEIGHT_SET     : chr  "Thoroughbred" "Thoroughbred" "Thoroughbred" "Thoroughbred" ...
##  $ FIELD_SIZE     : int  7 7 7 7 7 7 7 8 8 8 ...
##  $ TRACK_CONDITION: chr  "Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast" "Current Turf-Off, R 5,7,9 @
##  $ WETNESS_TIER   : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ DISTANCE       : int  550 550 550 550 550 550 550 700 700 700 ...
##  $ DIST_UNIT      : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
##  $ DIST_TEXT      : chr  "5 1/2 F" "5 1/2 F" "5 1/2 F" "5 1/2 F" ...
##  $ SURFACE        : chr  "Dirt" "Dirt" "Dirt" "Dirt" ...
##  $ RACE_TYPE      : chr  "MCL" "MCL" "MCL" "MCL" ...
##  $ RACE_CLASS     : chr  "MD 12500" "MD 12500" "MD 12500" "MD 12500" ...
##  $ PURSE          : int  25000 25000 25000 25000 25000 25000 25000 37000 37000 37000 ...
##  $ CLAIM_PRICE    : int  12500 12500 12500 12500 12500 12500 12500 20000 20000 20000 ...
##  $ SPEED_PAR      : int  72 72 72 72 72 72 72 76 76 76 ...
##  $ WEEK_RACES     : int  2 2 2 2 2 2 2 3 3 3 ...
##  $ MEET_RACES     : int  29 29 29 29 29 29 29 5 5 5 ...
##  $ PROGRAM        : int  2 4 8 7 3 1 6 5 6 8 ...
##  $ HORSE_NAME     : chr  "Senta Says" "Diamond's Honor" "Camm's Girl" "Starship Daytona" ...
##  $ POST           : int  2 4 8 7 3 1 6 5 6 8 ...
##  $ RUN_STYLE      : chr  "E/P" "E" "S" NA ...
##  $ RUN_STYLE_PTS  : int  5 6 0 4 0 0 0 2 1 4 ...
##  $ ODDS           : num  1.2 5 7 1.5 25 52 24 2.5 2.5 2.5 ...
##  $ ODDS_SRC       : chr  "LIVE" "LIVE" "LIVE" "LIVE" ...
##  $ C_SPEED        : num  67 64 69 58.6 54 ...
##  $ C_POWER        : num  110.2 105.6 99.3 101.3 97.4 ...
##  $ C_CLASS        : num  107 107 105 107 103 ...
##  $ C_DRIVER       : num  18.7 11.4 16.1 20.1 16.6 ...
##  $ C_TRAINER      : num  22.9 16 10 14.29 3.45 ...
##  $ C_EARLY        : num  91.5 91 82 86.5 81.5 81.5 85.6 89.5 84 92.5 ...
##  $ C_FINISH       : num  66 63 80 63 61 55 57.4 77 84 67 ...
##  $ C_RECENCY      : int  100 100 70 50 100 100 70 100 100 70 ...
##  $ C_MARKET       : num  45.45 16.67 12.5 40 3.85 ...
##  $ POST_BONUS     : num  2.16 0 0 0 2.16 5 0 0 0 0 ...
##  $ STYLE_BONUS    : num  0 2.97 0 0 0 0 0 0 0 0 ...
##  $ WET_BONUS      : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ FINAL_SCORE    : num  83.5 80.8 76.8 74.5 72.4 ...
##  $ RANK           : int  1 2 3 4 5 6 7 1 2 3 ...
##  $ Z_SCORE        : num  1.692 1.112 0.25 -0.234 -0.682 ...
##  $ WIN_PROB       : num  0.3282 0.2258 0.1332 0.1011 0.0796 ...
##  $ IMPUTED_COUNT  : int  0 0 1 2 0 0 12 0 2 0 ...
```

```
##  $ Is.Winner      : int  0 0 0 1 0 0 0 1 0 0 ...
##  $ In.Top3        : int  1 0 0 1 1 0 0 1 0 1 ...
```

```
names(data)
```

```
##  [1] "RACE_ID"         "WINNER"          "TOP3"            "DATE"
##  [5] "POST_TIME"       "SNAPSHOT_TIME"   "MIN_TO_POST"     "TRACK"
##  [9] "RACE_NUM"        "BREED"           "MODEL_VERSION"   "WEIGHT_SET"
## [13] "FIELD_SIZE"      "TRACK_CONDITION" "WETNESS_TIER"    "DISTANCE"
## [17] "DIST_UNIT"       "DIST_TEXT"       "SURFACE"         "RACE_TYPE"
## [21] "RACE_CLASS"      "PURSE"           "CLAIM_PRICE"     "SPEED_PAR"
## [25] "WEEK_RACES"      "MEET_RACES"      "PROGRAM"         "HORSE_NAME"
## [29] "POST"            "RUN_STYLE"       "RUN_STYLE_PTS"   "ODDS"
## [33] "ODDS_SRC"        "C_SPEED"         "C_POWER"         "C_CLASS"
## [37] "C_DRIVER"        "C_TRAINER"       "C_EARLY"         "C_FINISH"
## [41] "C_RECENCY"       "C_MARKET"        "POST_BONUS"      "STYLE_BONUS"
## [45] "WET_BONUS"       "FINAL_SCORE"     "RANK"            "Z_SCORE"
## [49] "WIN_PROB"        "IMPUTED_COUNT"   "Is.Winner"       "In.Top3"
```

```
colSums(is.na(data))
```

```
##         RACE_ID          WINNER            TOP3            DATE       POST_TIME
##               0               0               0               0               0
##   SNAPSHOT_TIME     MIN_TO_POST           TRACK        RACE_NUM           BREED
##               0              40               0               0               0
##   MODEL_VERSION      WEIGHT_SET      FIELD_SIZE TRACK_CONDITION    WETNESS_TIER
##               0               0               0               0               0
##        DISTANCE       DIST_UNIT       DIST_TEXT         SURFACE       RACE_TYPE
##              13              13               0               0               0
##      RACE_CLASS           PURSE     CLAIM_PRICE       SPEED_PAR      WEEK_RACES
##               0              13              13              13              13
##      MEET_RACES         PROGRAM      HORSE_NAME            POST       RUN_STYLE
##              13               0               0               0              44
##   RUN_STYLE_PTS            ODDS        ODDS_SRC         C_SPEED         C_POWER
##               0               0               0               0               0
##         C_CLASS        C_DRIVER       C_TRAINER         C_EARLY        C_FINISH
##               0               0               0               0               0
##       C_RECENCY        C_MARKET      POST_BONUS     STYLE_BONUS       WET_BONUS
##               0               0               0               0               0
##     FINAL_SCORE            RANK         Z_SCORE        WIN_PROB   IMPUTED_COUNT
##               0               0               0               0               0
##       Is.Winner         In.Top3
##               0               0
```

```
# ** DATA CLEANING ** ------------------------------------------------------------
# check class of each variable
sapply(data[core_components], class)
```

```
##   C_SPEED   C_POWER   C_CLASS  C_DRIVER C_TRAINER   C_EARLY  C_FINISH C_RECENCY
## "numeric" "numeric" "numeric" "numeric" "numeric" "numeric" "numeric" "integer"
##  C_MARKET
## "numeric"
```

```r
# convert recency to numeric and recheck class
data$C_RECENCY <- as.numeric(data$C_RECENCY)
sapply(data[core_components], class)
```

```
##   C_SPEED   C_POWER   C_CLASS  C_DRIVER C_TRAINER   C_EARLY  C_FINISH C_RECENCY
## "numeric" "numeric" "numeric" "numeric" "numeric" "numeric" "numeric" "numeric"
##  C_MARKET
## "numeric"
```

```r
# clean blanks
# blank bonus = no bonus awarded; blank MIN_TO_POST was clamped to 0 in the engine
bonus_cols <- c("POST_BONUS", "STYLE_BONUS", "WET_BONUS")
data[bonus_cols][is.na(data[bonus_cols])] <- 0
data$MIN_TO_POST[is.na(data$MIN_TO_POST)] <- 0

# rename dependent variables
data <- data |> rename(IS_WINNER = Is.Winner, IN_TOP3 = In.Top3)
str(data)
```

```
## 'data.frame':    188 obs. of  52 variables:
##  $ RACE_ID        : chr  "2026-10-02_Gulfstream Park_3" "2026-10-02_Gulfstream Park_3" "2026-10-02_Gu
##  $ WINNER         : int  7 7 7 7 7 7 7 5 5 5 ...
##  $ TOP3           : chr  "7,2,3" "7,2,3" "7,2,3" "7,2,3" ...
##  $ DATE           : int  46297 46297 46297 46297 46297 46297 46297 46297 46297 46297 ...
##  $ POST_TIME      : chr  "1:25pm" "1:25pm" "1:25pm" "1:25pm" ...
##  $ SNAPSHOT_TIME  : num  0.606 0.606 0.606 0.606 0.606 ...
##  $ MIN_TO_POST    : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ TRACK          : chr  "Gulfstream Park" "Gulfstream Park" "Gulfstream Park" "Gulfstream Park" ...
##  $ RACE_NUM       : int  3 3 3 3 3 3 3 1 1 1 ...
##  $ BREED          : chr  "Thoroughbred" "Thoroughbred" "Thoroughbred" "Thoroughbred" ...
##  $ MODEL_VERSION  : chr  "v2.2" "v2.2" "v2.2" "v2.2" ...
##  $ WEIGHT_SET     : chr  "Thoroughbred" "Thoroughbred" "Thoroughbred" "Thoroughbred" ...
##  $ FIELD_SIZE     : int  7 7 7 7 7 7 7 8 8 8 ...
##  $ TRACK_CONDITION: chr  "Current Turf-Off, R 5,7,9 @ 1m70y, Dirt-Fast" "Current Turf-Off, R 5,7,9 @
##  $ WETNESS_TIER   : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ DISTANCE       : int  550 550 550 550 550 550 550 700 700 700 ...
##  $ DIST_UNIT      : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
##  $ DIST_TEXT      : chr  "5 1/2 F" "5 1/2 F" "5 1/2 F" "5 1/2 F" ...
##  $ SURFACE        : chr  "Dirt" "Dirt" "Dirt" "Dirt" ...
##  $ RACE_TYPE      : chr  "MCL" "MCL" "MCL" "MCL" ...
##  $ RACE_CLASS     : chr  "MD 12500" "MD 12500" "MD 12500" "MD 12500" ...
##  $ PURSE          : int  25000 25000 25000 25000 25000 25000 25000 37000 37000 37000 ...
##  $ CLAIM_PRICE    : int  12500 12500 12500 12500 12500 12500 12500 20000 20000 20000 ...
##  $ SPEED_PAR      : int  72 72 72 72 72 72 72 76 76 76 ...
##  $ WEEK_RACES     : int  2 2 2 2 2 2 2 3 3 3 ...
##  $ MEET_RACES     : int  29 29 29 29 29 29 29 5 5 5 ...
##  $ PROGRAM        : int  2 4 8 7 3 1 6 5 6 8 ...
##  $ HORSE_NAME     : chr  "Senta Says" "Diamond's Honor" "Camm's Girl" "Starship Daytona" ...
##  $ POST           : int  2 4 8 7 3 1 6 5 6 8 ...
##  $ RUN_STYLE      : chr  "E/P" "E" "S" NA ...
##  $ RUN_STYLE_PTS  : int  5 6 0 4 0 0 0 2 1 4 ...
##  $ ODDS           : num  1.2 5 7 1.5 25 52 24 2.5 2.5 2.5 ...
##  $ ODDS_SRC       : chr  "LIVE" "LIVE" "LIVE" "LIVE" ...
```

```
##  $ C_SPEED        : num  67 64 69 58.6 54 ...
##  $ C_POWER        : num  110.2 105.6 99.3 101.3 97.4 ...
##  $ C_CLASS        : num  107 107 105 107 103 ...
##  $ C_DRIVER       : num  18.7 11.4 16.1 20.1 16.6 ...
##  $ C_TRAINER      : num  22.9 16 10 14.29 3.45 ...
##  $ C_EARLY        : num  91.5 91 82 86.5 81.5 81.5 85.6 89.5 84 92.5 ...
##  $ C_FINISH       : num  66 63 80 63 61 55 57.4 77 84 67 ...
##  $ C_RECENCY      : num  100 100 70 50 100 100 70 100 100 70 ...
##  $ C_MARKET       : num  45.45 16.67 12.5 40 3.85 ...
##  $ POST_BONUS     : num  2.16 0 0 0 2.16 5 0 0 0 0 ...
##  $ STYLE_BONUS    : num  0 2.97 0 0 0 0 0 0 0 0 ...
##  $ WET_BONUS      : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ FINAL_SCORE    : num  83.5 80.8 76.8 74.5 72.4 ...
##  $ RANK           : int  1 2 3 4 5 6 7 1 2 3 ...
##  $ Z_SCORE        : num  1.692 1.112 0.25 -0.234 -0.682 ...
##  $ WIN_PROB       : num  0.3282 0.2258 0.1332 0.1011 0.0796 ...
##  $ IMPUTED_COUNT  : int  0 0 1 2 0 0 12 0 2 0 ...
##  $ IS_WINNER      : int  0 0 0 1 0 0 0 1 0 0 ...
##  $ IN_TOP3        : int  1 0 0 1 1 0 0 1 0 1 ...
```

```
# races where a whole component is zero (never imputed)
zero_by_race <- data |>
  group_by(RACE_ID) |>
  summarise(across(all_of(core_components), ~ all(.x == 0)))
colSums(zero_by_race[core_components])
```

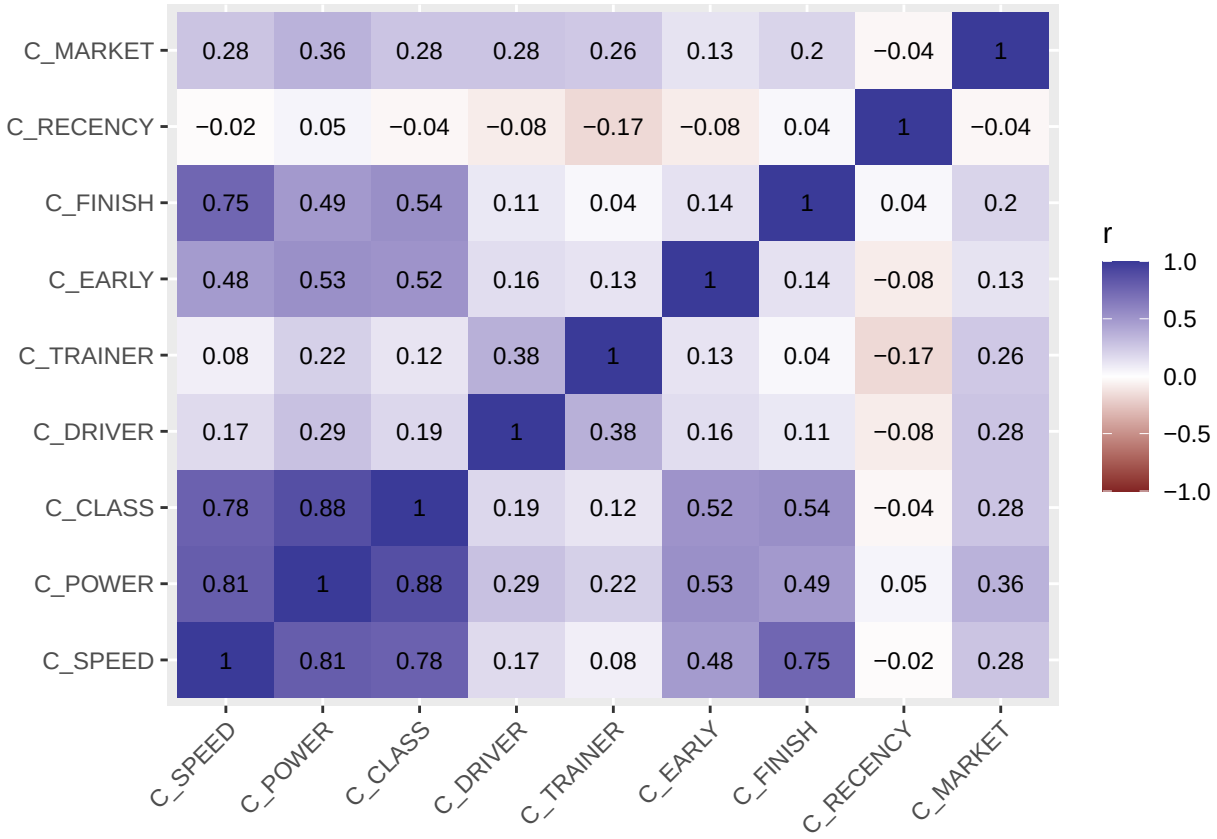
```
##   C_SPEED   C_POWER   C_CLASS  C_DRIVER C_TRAINER   C_EARLY  C_FINISH C_RECENCY
##         0         0         0         0         0         0         0         0
##  C_MARKET
##         0
```

```
# check winners and top 3 per race have all rows filled
data |> group_by(RACE_ID) |> summarise(w = sum(IS_WINNER), t3 = sum(IN_TOP3)) |>
  count(w, t3)
```

```
## # A tibble: 1 x 3
##       w    t3     n
##   <int> <int> <int>
## 1     1     3    25
```

```
# ** TESTING FOR MULTICOLLINEARITY ** ------------------------------------------
# variable heatmap (testing for multicollinearity)
cor_long <- as.data.frame(as.table(cor(data[core_components])))

ggplot(cor_long, aes(Var1, Var2, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = round(Freq, 2)), size = 3) +
  scale_fill_gradient2(limits = c(-1, 1)) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = NULL, y = NULL, fill = "r")
```




```
# VIF score for each variable
vif(lm(Z_SCORE ~ ., data = data[c("Z_SCORE", core_components)]))
```

```
##   C_SPEED   C_POWER   C_CLASS  C_DRIVER C_TRAINER   C_EARLY  C_FINISH C_RECENCY
##  6.521284  7.727802  5.478542  1.281776  1.296738  1.644992  3.082416  1.139430
##  C_MARKET
##  1.240118
```

```
# -- VIF scores pass with C_DIST removed. SPEED and POWER are still between 6 and 8 but not over 10 so


# ** RACE-LEVEL TABLE ** ------------------------------------------------------------
# check for ties at rank 1 and field size vs rows (any ties indicate an issue)
data |> group_by(RACE_ID) |> summarise(n_rank1 = sum(RANK == 1), rows = n(), field = first(FIELD_SIZE)) |>
  count(n_rank1, rows == field)
```

```
## # A tibble: 1 x 3
##   n_rank1 `rows == field`     n
##     <int> <lgl>           <int>
## 1       1 TRUE               25
```

```
# build one row per race
race_level <- data |>
  group_by(RACE_ID) |>
```

```
  summarise(field_size = first(FIELD_SIZE),
            top_won    = IS_WINNER[which.min(RANK)],
            top_in3    = IN_TOP3[which.min(RANK)],
            z_gap      = sort(Z_SCORE, decreasing = TRUE)[1] - sort(Z_SCORE, decreasing = TRUE)[2])

head(race_level)
```

```
## # A tibble: 6 x 5
##   RACE_ID                              field_size top_won top_in3  z_gap
##   <chr>                                     <int>   <int>   <int>  <dbl>
## 1 2026-10-02_Belmont_1                          8       1       1 0.0979
## 2 2026-10-02_Belmont_2                          7       0       1 1.02
## 3 2026-10-02_Gulfstream Park_3                  7       0       1 0.580
## 4 2026-10-05_Horseshoe Indianapolis_1           8       0       1 0.885
## 5 2026-10-05_Horseshoe Indianapolis_2           7       1       1 0.682
## 6 2026-10-05_Horseshoe Indianapolis_3           6       0       0 1.85
```

```
nrow(race_level)
```

```
## [1] 25
```

```
#-------------------------------------------------------------------------------
# **FORMATTING FOR ALL HYPOTHESIS TESTS BELOW** ---------------------------------
# H0: [null hypothesis]
# H1: [alternative hypothesis]
# alpha = 0.05
# test: [test type]

# [code to run test]
# decide(test$p.value)

# -- [results from tests]
# -- [reject / fail to reject H0]
# -- [real-world conclusion, written by hand]


# [next header/section after two blank lines]




# ** TOP PICK VS CHANCE ** ------------------------------------------------------
# H0: top pick wins at the chance rate (1 / field size)
# H1: top pick wins more often than chance
# alpha = 0.05
# test: one-sided exact binomial

# run test
test <- binom.test(sum(race_level$top_won), nrow(race_level),
                   p = mean(1 / race_level$field_size), alternative = "greater")
test
```

```
##
##  Exact binomial test
##
## data:  sum(race_level$top_won) and nrow(race_level)
## number of successes = 11, number of trials = 25, p-value = 0.0002293
## alternative hypothesis: true probability of success is greater than 0.1376032
## 95 percent confidence interval:
##  0.2698531 1.0000000
## sample estimates:
## probability of success
##                   0.44
```

```
decide(test$p.value)
```

```
## p-value = 0.0002293 -> reject H0
```

```
# -- p-value = 0.0002293
# -- reject H0
# -- top pick won 11 of 25 races (44%) vs about 14% by chance, roughly 3x chance probability


# ** TOP PICK IN TOP 3 VS CHANCE ** ------------------------------------------------------------
# H0: top pick finishes in the top 3 at the chance rate (3 / field size)
# H1: top pick finishes in the top 3 more often than chance
# alpha = 0.05
# test: one-sided exact binomial

# run test
test <- binom.test(sum(race_level$top_in3), nrow(race_level),
                   p = mean(3 / race_level$field_size), alternative = "greater")
test
```

```
##
##  Exact binomial test
##
## data:  sum(race_level$top_in3) and nrow(race_level)
## number of successes = 19, number of trials = 25, p-value = 0.0004544
## alternative hypothesis: true probability of success is greater than 0.4128095
## 95 percent confidence interval:
##  0.58048 1.00000
## sample estimates:
## probability of success
##                   0.76
```

```
decide(test$p.value)
```

```
## p-value = 0.0004544 -> reject H0
```

```
# -- p-value = 0.0004544
# -- reject H0
# -- top pick finished in the top 3 in 19 of 25 races (76%) vs about 41% by chance, nearly double chanc
```

```r
# ** SPACING: Z GAP VS TOP PICK WINNING ** ------------------------------------------------------------
# H0: z gap has no relationship with the top pick winning (slope = 0)
# H1: z gap is related to the top pick winning (slope != 0)
# alpha = 0.05
# test: logistic regression, Wald test on the z_gap slope

gap_model <- glm(top_won ~ z_gap, data = race_level, family = binomial)
summary(gap_model)
```

```
##
## Call:
## glm(formula = top_won ~ z_gap, family = binomial, data = race_level)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.3761     0.7510  -0.501    0.617
## z_gap         0.1453     0.6804   0.214    0.831
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 34.296  on 24  degrees of freedom
## Residual deviance: 34.251  on 23  degrees of freedom
## AIC: 38.251
##
## Number of Fisher Scoring iterations: 4
```

```r
# odds ratio per 1 unit of z gap, with 95% CI
exp(cbind(OR = coef(gap_model), confint(gap_model)))
```

```
## Waiting for profiling to be done...
```

```
##                    OR     2.5 %   97.5 %
## (Intercept) 0.6865273 0.1472185 2.985469
## z_gap       1.1564225 0.2952856 4.572973
```

```r
# average z gap when top pick won vs lost
race_level |> group_by(top_won) |> summarise(n = n(), mean_gap = mean(z_gap))
```

```
## # A tibble: 2 x 3
##   top_won     n mean_gap
##     <int> <int>    <dbl>
## 1       0    14    0.903
## 2       1    11    0.954
```

```r
# -- p-value = 0.831
# -- fail to reject H0
# -- z gap showed no detectable link to the top pick winning: avg gap was 0.95 in wins vs 0.90 in losse


# ** WINNERS VS NON-WINNERS: Z SCORE ** ---------------------------------------------------------------
```

```r
# H0: mean z score is the same for winners and non-winners
# H1: winners have a higher mean z score than non-winners
# alpha = 0.05
# test: one-sided Welch t-test, with Wilcoxon rank-sum as a check

# group sizes and means
data |> group_by(IS_WINNER) |> summarise(n = n(), mean_z = mean(Z_SCORE), sd_z = sd(Z_SCORE))
```

```
## # A tibble: 2 x 4
##   IS_WINNER     n mean_z  sd_z
##       <int> <int>  <dbl> <dbl>
## 1         0   163 -0.132 0.944
## 2         1    25  0.858 0.966
```

```r
# run test
t.test(Z_SCORE ~ IS_WINNER, data = data, alternative = "less")
```

```
##
##  Welch Two Sample t-test
##
## data:  Z_SCORE by IS_WINNER
## t = -4.7834, df = 31.449, p-value = 1.934e-05
## alternative hypothesis: true difference in means between group 0 and group 1 is less than 0
## 95 percent confidence interval:
##        -Inf -0.6388561
## sample estimates:
## mean in group 0 mean in group 1
##      -0.1315681       0.8578360
```

```r
wilcox.test(Z_SCORE ~ IS_WINNER, data = data, alternative = "less")
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  Z_SCORE by IS_WINNER
## W = 928, p-value = 6.002e-06
## alternative hypothesis: true location shift is less than 0
```

```r
# effect size (cohen's d)
cohens_d(data$Z_SCORE, data$IS_WINNER)
```

```
## [1] 1.044854
```

```r
# -- cohen's d = 1.044854
# -- large effect (0.8+ is conventionally large)
# -- p-value = 1.934e-05 (Welch), 6.002e-06 (Wilcoxon)
# -- reject H0
# -- winners averaged a z score of +0.86 vs -0.13 for non-winners, about 1 standard deviation higher
```

```r
# ** PER-VARIABLE COMPARISON: RAW ** ------------------------------------------------------------
# H0: the component has the same mean for winners and non-winners
# H1: the component has a different mean for winners and non-winners
# alpha = 0.05 (applied to Holm and BH adjusted p-values)
# test: Welch t-test per component, with Holm and BH multiple-testing corrections

# run test for each component (winner)
raw_win <- data.frame(
  component = core_components,
  d = sapply(core_components, function(v) cohens_d(data[[v]], data$IS_WINNER)),
  p = sapply(core_components, function(v) t.test(data[[v]] ~ data$IS_WINNER)$p.value)
)
raw_win$p_holm <- p.adjust(raw_win$p, method = "holm")
raw_win$p_bh   <- p.adjust(raw_win$p, method = "BH")
raw_win[order(-abs(raw_win$d)), ]
```

```
##           component           d            p      p_holm        p_bh
## C_MARKET   C_MARKET  1.14229650 0.0005471769 0.004924592 0.004924592
## C_CLASS     C_CLASS  0.32848254 0.1234168609 0.987334887 0.382352017
## C_DRIVER   C_DRIVER  0.32472616 0.1699342297 1.000000000 0.382352017
## C_SPEED     C_SPEED  0.29981883 0.2563429746 1.000000000 0.384514462
## C_EARLY     C_EARLY  0.29911562 0.2498006453 1.000000000 0.384514462
## C_POWER     C_POWER  0.28865502 0.1575559370 1.000000000 0.382352017
## C_FINISH   C_FINISH  0.16813195 0.5183623533 1.000000000 0.666465883
## C_TRAINER C_TRAINER  0.10584394 0.6354517688 1.000000000 0.714883240
## C_RECENCY C_RECENCY -0.06446817 0.7757210535 1.000000000 0.775721054
```

```r
# run test for each component (top 3)
raw_top3 <- data.frame(
  component = core_components,
  d = sapply(core_components, function(v) cohens_d(data[[v]], data$IN_TOP3)),
  p = sapply(core_components, function(v) t.test(data[[v]] ~ data$IN_TOP3)$p.value)
)
raw_top3$p_holm <- p.adjust(raw_top3$p, method = "holm")
raw_top3$p_bh   <- p.adjust(raw_top3$p, method = "BH")
raw_top3[order(-abs(raw_top3$d)), ]
```

```
##           component           d            p       p_holm         p_bh
## C_MARKET   C_MARKET  0.81122088 3.247263e-06 2.922537e-05 2.922537e-05
## C_POWER     C_POWER  0.29389567 5.177724e-02 4.142179e-01 2.088912e-01
## C_CLASS     C_CLASS  0.26814518 6.963041e-02 4.874129e-01 2.088912e-01
## C_SPEED     C_SPEED  0.22301246 1.395046e-01 8.370274e-01 3.138853e-01
## C_DRIVER   C_DRIVER  0.17130396 2.493330e-01 1.000000e+00 4.487995e-01
## C_EARLY     C_EARLY  0.12348477 4.276573e-01 1.000000e+00 5.812119e-01
## C_TRAINER C_TRAINER  0.09882284 5.166328e-01 1.000000e+00 5.812119e-01
## C_FINISH   C_FINISH  0.09687607 5.148360e-01 1.000000e+00 5.812119e-01
## C_RECENCY C_RECENCY -0.00116449 9.937926e-01 1.000000e+00 9.937926e-01
```

```r
#FOR IS_WINNER:
# -- C_MARKET: d = 1.14, raw p = 0.0005, Holm/BH p = 0.0049
# -- reject H0 for C_MARKET only; fail to reject H0 for the other 8 components
# -- winners had a much higher market component (the favorites win more); no other component separated
```

```
#FOR IS_TOP3:
# -- C_MARKET: d = 0.81, raw p = 3.2e-06, Holm/BH p = 2.9e-05
# -- reject H0 for C_MARKET only; fail to reject H0 for the other 8 components
# -- market again separates top-3 finishers best; power (d = 0.29, p = 0.052) and class (d = 0.27, p =


# ** PER-VARIABLE COMPARISON: WITHIN-RACE ** ------------------------------------------------------
# H0: the race-centered component has the same mean for winners (or top 3) and others
# H1: the race-centered component has a different mean for winners (or top 3) and others
# alpha = 0.05 (applied to Holm and BH adjusted p-values)
# test: Welch t-test per race-centered component, with Holm and BH corrections

# center each component within race
data_c <- data |>
  group_by(RACE_ID) |>
  mutate(across(all_of(core_components), ~ .x - mean(.x))) |>
  ungroup()

# comparison function
compare_components <- function(df, outcome) {
  out <- data.frame(
    component = core_components,
    d = sapply(core_components, function(v) cohens_d(df[[v]], df[[outcome]])),
    p = sapply(core_components, function(v) t.test(df[[v]] ~ df[[outcome]])$p.value)
  )
  out$p_holm <- p.adjust(out$p, method = "holm")
  out$p_bh   <- p.adjust(out$p, method = "BH")
  out[order(-abs(out$d)), ]
}

# run test (winner and top 3)
compare_components(data_c, "IS_WINNER")
```

```
##           component           d            p      p_holm        p_bh
## C_MARKET   C_MARKET  1.11593416 0.0004818062 0.004161599 0.002168128
## C_CLASS     C_CLASS  0.74790888 0.0004623999 0.004161599 0.002168128
## C_SPEED     C_SPEED  0.58918087 0.0284726039 0.170835623 0.064063359
## C_EARLY     C_EARLY  0.56773963 0.0380221312 0.190110656 0.068439836
## C_POWER     C_POWER  0.52377535 0.0097089622 0.067962735 0.029126887
## C_DRIVER   C_DRIVER  0.38528416 0.0744958860 0.297983544 0.111743829
## C_FINISH   C_FINISH  0.23690059 0.3603038621 1.000000000 0.463247823
## C_TRAINER C_TRAINER  0.12503668 0.5708125890 1.000000000 0.642164163
## C_RECENCY C_RECENCY -0.06767684 0.7518661430 1.000000000 0.751866143
```

```
compare_components(data_c, "IN_TOP3")
```

```
##           component           d            p       p_holm         p_bh
## C_MARKET   C_MARKET 0.750208532 1.439131e-05 1.151305e-04 6.476089e-05
## C_CLASS     C_CLASS 0.641822479 1.055122e-05 9.496094e-05 6.476089e-05
## C_POWER     C_POWER 0.537822596 2.635454e-04 1.844818e-03 7.906361e-04
## C_SPEED     C_SPEED 0.407562559 8.560423e-03 5.136254e-02 1.926095e-02
## C_EARLY     C_EARLY 0.284558780 6.957476e-02 3.478738e-01 1.252346e-01
```

```
## C_DRIVER   C_DRIVER 0.223472312 1.268077e-01 5.072306e-01 1.902115e-01
## C_FINISH   C_FINISH 0.144615610 3.351174e-01 1.000000e+00 4.308653e-01
## C_TRAINER C_TRAINER 0.120076802 4.312528e-01 1.000000e+00 4.851594e-01
## C_RECENCY C_RECENCY 0.002427315 9.867836e-01 1.000000e+00 9.867836e-01
```

```
# -- winner: C_MARKET d = 1.12 and C_CLASS d = 0.75 survive Holm (p = 0.004); speed, early and power ar
# -- top 3: C_MARKET, C_CLASS and C_POWER survive Holm (p = 0.0001, 0.0001, 0.002); speed is borderline
# -- reject H0 for market, class and power; fail to reject H0 for early, driver, finish, trainer and re
# -- once each horse is compared only to its own field, class and power clearly separate winners and to


# ** CONDITIONAL LOGIT: TOP 3 ** ----------------------------------------------------------------
# H0: all component coefficients = 0 (components do not predict top 3 within a race)
# H1: at least one component coefficient != 0
# alpha = 0.05
# test: conditional logistic regression (clogit), likelihood-ratio and Wald tests

# standardize components (centered values divided by their sd)
data_s <- data_c |> mutate(across(all_of(core_components), ~ .x / sd(.x)))

# fit model
m_top3 <- clogit(reformulate(c(core_components, "strata(RACE_ID)"), response = "IN_TOP3"),
                 data = data_s)
summary(m_top3)
```

```
## Call:
## coxph(formula = Surv(rep(1, 188L), IN_TOP3) ~ C_SPEED + C_POWER +
##     C_CLASS + C_DRIVER + C_TRAINER + C_EARLY + C_FINISH + C_RECENCY +
##     C_MARKET + strata(RACE_ID), data = data_s, method = "exact")
##
##   n= 188, number of events= 75
##
##               coef exp(coef) se(coef)      z Pr(>|z|)
## C_SPEED   -0.03044   0.97002  0.35817 -0.085   0.9323
## C_POWER   -0.19564   0.82231  0.31083 -0.629   0.5291
## C_CLASS    0.48444   1.62326  0.28517  1.699   0.0894 .
## C_DRIVER   0.02811   1.02851  0.17676  0.159   0.8737
## C_TRAINER -0.10250   0.90257  0.17265 -0.594   0.5527
## C_EARLY    0.02423   1.02453  0.22564  0.107   0.9145
## C_FINISH  -0.17940   0.83577  0.30815 -0.582   0.5605
## C_RECENCY  0.09780   1.10274  0.16732  0.584   0.5589
## C_MARKET   0.69553   2.00478  0.27021  2.574   0.0101 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##           exp(coef) exp(-coef) lower .95 upper .95
## C_SPEED      0.9700     1.0309    0.4807     1.957
## C_POWER      0.8223     1.2161    0.4472     1.512
## C_CLASS      1.6233     0.6160    0.9282     2.839
## C_DRIVER     1.0285     0.9723    0.7274     1.454
## C_TRAINER    0.9026     1.1079    0.6435     1.266
## C_EARLY      1.0245     0.9761    0.6583     1.594
## C_FINISH     0.8358     1.1965    0.4569     1.529
## C_RECENCY    1.1027     0.9068    0.7944     1.531
```

```
## C_MARKET     2.0048     0.4988    1.1805     3.405
##
## Concordance= 0.714  (se = 0.045 )
## Likelihood ratio test= 25.65  on 9 df,   p=0.002
## Wald test            = 18.77  on 9 df,   p=0.03
## Score (logrank) test = 22.98  on 9 df,   p=0.006
```

```
# -- p-value = 0.002 (likelihood-ratio test, 9 df)
# -- reject H0
# -- the 9 components together predict which horses finish top 3 within a race (concordance 0.71)
# -- individually only C_MARKET is significant (coef 0.70, p = 0.010); C_CLASS is borderline (coef 0.48
```

```
# ** CONDITIONAL LOGIT: BEYOND THE MARKET ** ------------------------------------------------------
# H0: the 8 non-market components add nothing beyond C_MARKET (their coefficients are all 0)
# H1: at least one non-market component adds information beyond C_MARKET
# alpha = 0.05
# test: likelihood-ratio test comparing market-only vs full clogit

# fit market-only model
m_mkt <- clogit(IN_TOP3 ~ C_MARKET + strata(RACE_ID), data = data_s)

# run test
anova(m_mkt, m_top3)
```

```
## Analysis of Deviance Table
##  Cox model: response is  Surv(rep(1, 188L), IN_TOP3)
##  Model 1: ~ C_MARKET + strata(RACE_ID)
##  Model 2: ~ C_SPEED + C_POWER + C_CLASS + C_DRIVER + C_TRAINER + C_EARLY + C_FINISH + C_RECENCY + C_
##    loglik  Chisq Df Pr(>|Chi|)
## 1 -82.721
## 2 -80.545 4.3516  8     0.8241
```

```
# -- p-value = 0.824 (chi-square = 4.35 on 8 df)
# -- fail to reject H0
# -- the 8 non-market components did not improve the top-3 fit beyond C_MARKET alone (log-likelihood -8
```

```
# ** OVERLAP WITH MARKET ** -----------------------------------------------------------
# within-race correlation of each component with C_MARKET (descriptive, not a hypothesis test)
cor(data_c[core_components])[, "C_MARKET"]
```

```
##     C_SPEED     C_POWER     C_CLASS    C_DRIVER   C_TRAINER     C_EARLY
##  0.54816439  0.66997983  0.62777093  0.33716903  0.29963293  0.28224203
##    C_FINISH   C_RECENCY    C_MARKET
##  0.29079832 -0.04320494  1.00000000
```

```
# -- (record the conclusion after running)
```

```
# ** CONDITIONAL LOGIT: BEYOND THE MARKET (WINNER) ** ---------------------------------------------
# H0: the 8 non-market components add nothing beyond C_MARKET for predicting the winner
```

```r
# H1: at least one non-market component adds information beyond C_MARKET for predicting the winner
# alpha = 0.05
# test: likelihood-ratio test comparing market-only vs full clogit (winner)

# fit models
m_win_mkt  <- clogit(IS_WINNER ~ C_MARKET + strata(RACE_ID), data = data_s)
m_win_full <- clogit(reformulate(c(core_components, "strata(RACE_ID)"), response = "IS_WINNER"),
                     data = data_s)
summary(m_win_full)
```

```
## Call:
## coxph(formula = Surv(rep(1, 188L), IS_WINNER) ~ C_SPEED + C_POWER +
##     C_CLASS + C_DRIVER + C_TRAINER + C_EARLY + C_FINISH + C_RECENCY +
##     C_MARKET + strata(RACE_ID), data = data_s, method = "exact")
##
##   n= 188, number of events= 25
##
##               coef exp(coef) se(coef)      z Pr(>|z|)
## C_SPEED   -0.0444    0.9566   0.4805 -0.092   0.9264
## C_POWER   -0.7085    0.4924   0.4129 -1.716   0.0862 .
## C_CLASS    0.6358    1.8884   0.4073  1.561   0.1185
## C_DRIVER   0.3083    1.3611   0.2830  1.089   0.2761
## C_TRAINER -0.1153    0.8911   0.2818 -0.409   0.6825
## C_EARLY    0.4203    1.5224   0.3471  1.211   0.2259
## C_FINISH   0.1018    1.1071   0.4517  0.225   0.8217
## C_RECENCY  0.1847    1.2028   0.2374  0.778   0.4367
## C_MARKET   0.6169    1.8531   0.3141  1.964   0.0496 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##           exp(coef) exp(-coef) lower .95 upper .95
## C_SPEED      0.9566     1.0454    0.3730     2.453
## C_POWER      0.4924     2.0309    0.2192     1.106
## C_CLASS      1.8885     0.5295    0.8500     4.196
## C_DRIVER     1.3611     0.7347    0.7816     2.370
## C_TRAINER    0.8911     1.1222    0.5129     1.548
## C_EARLY      1.5224     0.6568    0.7711     3.006
## C_FINISH     1.1071     0.9032    0.4568     2.683
## C_RECENCY    1.2028     0.8314    0.7553     1.915
## C_MARKET     1.8531     0.5396    1.0011     3.430
##
## Concordance= 0.822  (se = 0.056 )
## Likelihood ratio test= 22  on 9 df,   p=0.009
## Wald test            = 16.74  on 9 df,   p=0.05
## Score (logrank) test = 25.33  on 9 df,   p=0.003
```

```r
# run test
anova(m_win_mkt, m_win_full)
```

```
## Analysis of Deviance Table
##  Cox model: response is  Surv(rep(1, 188L), IS_WINNER)
##  Model 1: ~ C_MARKET + strata(RACE_ID)
##  Model 2: ~ C_SPEED + C_POWER + C_CLASS + C_DRIVER + C_TRAINER + C_EARLY + C_FINISH + C_RECENCY + C_
```

```
##    loglik  Chisq Df Pr(>|Chi|)
## 1 -42.448
## 2 -38.993 6.9084  8     0.5465
```

```
# -- (record p-value, decision and conclusion after running)


# ** Z SCORE SLOPE VS ENGINE K ** ------------------------------------------------------------
# H0: the z score slope for winning equals the engine's assumed softmax k (0.70)
# H1: the z score slope differs from 0.70
# alpha = 0.05
# test: clogit on Z_SCORE (within race), Wald test of slope vs 0.70

k_engine <- 0.70

# fit model (Z_SCORE is already within-race, so no centering needed)
m_slope <- clogit(IS_WINNER ~ Z_SCORE + strata(RACE_ID), data = data)
summary(m_slope)
```

```
## Call:
## coxph(formula = Surv(rep(1, 188L), IS_WINNER) ~ Z_SCORE + strata(RACE_ID),
##     data = data, method = "exact")
##
##   n= 188, number of events= 25
##
##           coef exp(coef) se(coef)     z Pr(>|z|)
## Z_SCORE 0.8760    2.4014   0.2178 4.022 5.76e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##         exp(coef) exp(-coef) lower .95 upper .95
## Z_SCORE     2.401     0.4164     1.567      3.68
##
## Concordance= 0.767  (se = 0.068 )
## Likelihood ratio test= 18.32  on 1 df,   p=2e-05
## Wald test            = 16.18  on 1 df,   p=6e-05
## Score (logrank) test = 18.4  on 1 df,   p=2e-05
```

```
slope    <- unname(coef(m_slope)["Z_SCORE"])
slope_se <- sqrt(vcov(m_slope)["Z_SCORE", "Z_SCORE"])
slope_p  <- 2 * pnorm(-abs((slope - k_engine) / slope_se))
slope_ci <- slope + c(-1.96, 1.96) * slope_se
slope_ci
```

```
## [1] 0.449179 1.302903
```

```
decide(slope_p)
```

```
## p-value = 0.4189 -> fail to reject H0
```

```
# -- fitted slope = 0.876 (95% CI 0.45 to 1.30), p-value vs 0.70 = 0.4189
# -- fail to reject H0
# -- the fitted slope is consistent with the engine's 0.70; the data can't tell them apart yet (25 winn


# ** CALIBRATION: WIN_PROB VS ACTUAL WIN RATE ** ------------------------------------------------------
# H0: the engine's win probabilities are calibrated (predicted = actual win rate in each bucket)
# H1: the engine's win probabilities are not calibrated
# alpha = 0.05
# test: hosmer-lemeshow chi-square across probability buckets

# bucket win probabilities and compare with actual wins
calib <- data |>
  mutate(prob_bin = cut(WIN_PROB, breaks = c(0, 0.10, 0.20, 0.30, 1), include.lowest = TRUE)) |>
  group_by(prob_bin) |>
  summarise(n = n(), predicted = mean(WIN_PROB), actual = mean(IS_WINNER),
            exp_wins = sum(WIN_PROB), act_wins = sum(IS_WINNER))
calib
```

```
## # A tibble: 4 x 6
##   prob_bin      n predicted actual exp_wins act_wins
##   <fct>     <int>     <dbl>  <dbl>    <dbl>    <int>
## 1 [0,0.1]      85    0.0684 0.0235     5.82        2
## 2 (0.1,0.2]    72    0.136  0.167      9.77       12
## 3 (0.2,0.3]    17    0.240  0.235      4.08        4
## 4 (0.3,1]      14    0.381  0.5        5.33        7
```

```
# run test
hl_stat <- sum((calib$act_wins - calib$exp_wins)^2 / (calib$n * calib$predicted * (1 - calib$predicted))
hl_df   <- nrow(calib) - 2
hl_p    <- pchisq(hl_stat, hl_df, lower.tail = FALSE)
hl_stat
```

```
## [1] 4.126166
```

```
hl_df
```

```
## [1] 2
```

```
hl_p
```

```
## [1] 0.1270616
```

```
decide(hl_p)
```

```
## p-value = 0.1271 -> fail to reject H0
```

```
# -- hosmer-lemeshow chi-square = 4.13 on 2 df, p-value = 0.1271
# -- fail to reject H0
# -- no significant miscalibration, but the table shows a pattern: the top bucket (>30%) won 50% vs 38%
```

```r
# ** NEAR-TIE THRESHOLD: Z GAP 0.41 ** ------------------------------------------------------------
# H0: the top pick wins at the same rate in near-tie races (z gap < 0.41) and clear races (z gap >= 0.4
# H1: the top pick wins more often in clear races than in near-tie races
# alpha = 0.05
# test: one-sided fisher exact test

# races in each group and top pick wins
race_level$clear <- race_level$z_gap >= 0.41
table(race_level$clear, race_level$top_won)
```

```
##
##          0  1
##   FALSE  1  3
##   TRUE  13  8
```

```r
# run test
fisher.test(table(race_level$clear, race_level$top_won), alternative = "greater")
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  table(race_level$clear, race_level$top_won)
## p-value = 0.9739
## alternative hypothesis: true odds ratio is greater than 1
## 95 percent confidence interval:
##  0.007393122         Inf
## sample estimates:
## odds ratio
##  0.2186967
```

```r
# -- near-tie races (z gap < 0.41): top pick won 3 of 4; clear races: top pick won 8 of 21
# -- p-value = 0.9739
# -- fail to reject H0
# -- no support for the near-tie rule, but only 4 races fall below 0.41, so this test can't say anythin


# ** CLASS VS POWER OVERLAP ** ------------------------------------------------------------
# H0: C_POWER adds nothing beyond C_CLASS (given the market); and C_CLASS adds nothing beyond C_POWER (
# H1: each one adds information beyond the other
# alpha = 0.05
# test: likelihood-ratio tests on nested clogit models (top 3)

# within-race correlation of class and power
cor(data_c$C_CLASS, data_c$C_POWER)
```

```
## [1] 0.7365284
```

```r
# fit models
m_cm  <- clogit(IN_TOP3 ~ C_MARKET + C_CLASS + strata(RACE_ID), data = data_s)
m_pm  <- clogit(IN_TOP3 ~ C_MARKET + C_POWER + strata(RACE_ID), data = data_s)
```

```r
m_cpm <- clogit(IN_TOP3 ~ C_MARKET + C_CLASS + C_POWER + strata(RACE_ID), data = data_s)

# run test: does power add beyond class?
anova(m_cm, m_cpm)
```

```
## Analysis of Deviance Table
##  Cox model: response is  Surv(rep(1, 188L), IN_TOP3)
##  Model 1: ~ C_MARKET + C_CLASS + strata(RACE_ID)
##  Model 2: ~ C_MARKET + C_CLASS + C_POWER + strata(RACE_ID)
##    loglik  Chisq Df Pr(>|Chi|)
## 1 -81.618
## 2 -81.509 0.2171  1     0.6413
```

```r
# run test: does class add beyond power?
anova(m_pm, m_cpm)
```

```
## Analysis of Deviance Table
##  Cox model: response is  Surv(rep(1, 188L), IN_TOP3)
##  Model 1: ~ C_MARKET + C_POWER + strata(RACE_ID)
##  Model 2: ~ C_MARKET + C_CLASS + C_POWER + strata(RACE_ID)
##    loglik  Chisq Df Pr(>|Chi|)
## 1 -82.636
## 2 -81.509 2.2524  1     0.1334
```

```r
# compare model fit (lower AIC = better)
AIC(m_cm, m_pm, m_cpm)
```

```
##       df      AIC
## m_cm   2 167.2358
## m_pm   2 169.2711
## m_cpm  3 169.0187
```

```r
# -- power adds beyond class: p-value = 0.6413; class adds beyond power: p-value = 0.1334
# -- fail to reject H0 (both directions)
# -- neither component adds significant information beyond the other once the market is in; class alone


# ** FITTED VS CURRENT WEIGHTS ** ------------------------------------------------------------
# descriptive comparison (not a hypothesis test); exploratory until there is a much larger sample

# current engine weights (handi.js weightsDict, v2.4): Thoroughbred
w_current <- c(0.28, 0.09, 0.16, 0.04, 0.07, 0.15, 0.10, 0.03, 0.03)
# Special: c(0.25, 0.08, 0.15, 0.04, 0.08, 0.14, 0.11, 0.03, 0.09)
names(w_current) <- core_components

# within-race sd of each component (puts weights on the same scale as the standardized coefficients)
sd_within <- sapply(data_c[core_components], sd)

# current effective importance (weight x within-race sd) as a share of the total
cur_share <- w_current * sd_within / sum(w_current * sd_within)
```

```r
# fitted standardized coefficients from the top 3 model (negatives set to 0 for the share)
fit_beta  <- coef(m_top3)[core_components]
fit_se    <- sqrt(diag(vcov(m_top3)))[core_components]
fit_share <- pmax(fit_beta, 0) / sum(pmax(fit_beta, 0))

# compare
weights_compare <- data.frame(component = core_components,
                              current_weight = w_current,
                              current_share  = round(cur_share, 3),
                              fitted_beta    = round(fit_beta, 3),
                              fitted_se      = round(fit_se, 3),
                              fitted_share   = round(fit_share, 3))
rownames(weights_compare) <- NULL
weights_compare
```

```
##   component current_weight current_share fitted_beta fitted_se fitted_share
## 1   C_SPEED           0.28         0.308      -0.030     0.358        0.000
## 2   C_POWER           0.09         0.101      -0.196     0.311        0.000
## 3   C_CLASS           0.16         0.046       0.484     0.285        0.364
## 4  C_DRIVER           0.04         0.030       0.028     0.177        0.021
## 5 C_TRAINER           0.07         0.058      -0.103     0.173        0.000
## 6   C_EARLY           0.15         0.152       0.024     0.226        0.018
## 7  C_FINISH           0.10         0.155      -0.179     0.308        0.000
## 8 C_RECENCY           0.03         0.075       0.098     0.167        0.074
## 9  C_MARKET           0.03         0.076       0.696     0.270        0.523
```

```r
# -- C_CLASS: fitted beta 0.48 (se 0.29), fitted share 36% vs current effective share 4.6%
# -- C_MARKET: fitted beta 0.70 (se 0.27), fitted share 52% vs current effective share 7.6%
# -- the other 7 components have fitted betas smaller than 1 to 2 standard errors, so the data can't sa
# -- speed, power, finish and trainer have negative betas, which reflects overlap between components an


# ** BONUS TERMS ** ------------------------------------------------------------
# H0: post and style bonuses add nothing beyond the 9 components (their coefficients are 0)
# H1: at least one bonus adds information beyond the 9 components
# alpha = 0.05
# test: likelihood-ratio test, components vs components + bonuses (clogit, top 3)

# check which bonuses vary (a bonus that is the same for every horse can't be tested)
sapply(data[bonus_cols], function(x) length(unique(x)))
```

```
##  POST_BONUS STYLE_BONUS   WET_BONUS
##          15          30           1
```

```r
# fit model with bonuses
m_bonus <- clogit(reformulate(c(core_components, "POST_BONUS", "STYLE_BONUS", "strata(RACE_ID)"),
                              response = "IN_TOP3"), data = data_s)
summary(m_bonus)
```

```
## Call:
## coxph(formula = Surv(rep(1, 188L), IN_TOP3) ~ C_SPEED + C_POWER +
##     C_CLASS + C_DRIVER + C_TRAINER + C_EARLY + C_FINISH + C_RECENCY +
```

```
##     C_MARKET + POST_BONUS + STYLE_BONUS + strata(RACE_ID), data = data_s,
##     method = "exact")
##
##   n= 188, number of events= 75
##
##                 coef exp(coef) se(coef)      z Pr(>|z|)
## C_SPEED      0.02414   1.02443  0.36701  0.066   0.9476
## C_POWER     -0.16299   0.84960  0.31336 -0.520   0.6030
## C_CLASS      0.46746   1.59593  0.28898  1.618   0.1057
## C_DRIVER     0.04561   1.04666  0.17960  0.254   0.7995
## C_TRAINER   -0.09117   0.91286  0.17349 -0.526   0.5992
## C_EARLY      0.01675   1.01689  0.23336  0.072   0.9428
## C_FINISH    -0.19361   0.82398  0.31567 -0.613   0.5397
## C_RECENCY    0.08772   1.09169  0.16994  0.516   0.6057
## C_MARKET     0.63187   1.88113  0.27309  2.314   0.0207 *
## POST_BONUS   0.24064   1.27207  0.18812  1.279   0.2008
## STYLE_BONUS  0.03420   1.03479  0.19836  0.172   0.8631
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##             exp(coef) exp(-coef) lower .95 upper .95
## C_SPEED        1.0244     0.9762    0.4990     2.103
## C_POWER        0.8496     1.1770    0.4597     1.570
## C_CLASS        1.5959     0.6266    0.9058     2.812
## C_DRIVER       1.0467     0.9554    0.7361     1.488
## C_TRAINER      0.9129     1.0955    0.6497     1.283
## C_EARLY        1.0169     0.9834    0.6436     1.607
## C_FINISH       0.8240     1.2136    0.4438     1.530
## C_RECENCY      1.0917     0.9160    0.7824     1.523
## C_MARKET       1.8811     0.5316    1.1014     3.213
## POST_BONUS     1.2721     0.7861    0.8798     1.839
## STYLE_BONUS    1.0348     0.9664    0.7015     1.526
##
## Concordance= 0.732  (se = 0.043 )
## Likelihood ratio test= 27.44  on 11 df,   p=0.004
## Wald test            = 20.18  on 11 df,   p=0.04
## Score (logrank) test = 24.89  on 11 df,   p=0.009
```

```
# run test
anova(m_top3, m_bonus)
```

```
## Analysis of Deviance Table
##  Cox model: response is  Surv(rep(1, 188L), IN_TOP3)
##  Model 1: ~ C_SPEED + C_POWER + C_CLASS + C_DRIVER + C_TRAINER + C_EARLY + C_FINISH + C_RECENCY + C_
##  Model 2: ~ C_SPEED + C_POWER + C_CLASS + C_DRIVER + C_TRAINER + C_EARLY + C_FINISH + C_RECENCY + C_
##    loglik  Chisq Df Pr(>|Chi|)
## 1 -80.545
## 2 -79.649 1.7916  2     0.4083
```

```
# -- chi-square = 1.79 on 2 df, p-value = 0.4083
# -- fail to reject H0
# -- the post and style bonuses did not improve the top-3 fit beyond the 9 components (log-likelihood -
```

```r
# ** OUT-OF-SAMPLE: LEAVE-ONE-RACE-OUT ** ------------------------------------------------------------
# H0: the engine and the fitted 9-component model have the same mean log-loss on held-out races
# H1: the mean log-loss differs between the engine and the fitted 9-component model
# alpha = 0.05
# test: paired t-test on per-race log-loss

races <- sort(unique(data_s$RACE_ID))

# leave-one-race-out log-loss for a set of components (fit on 24 races, predict the held-out race)
cv_fit <- function(terms) {
  sapply(races, function(r) {
    train <- data_s[data_s$RACE_ID != r, ]
    test  <- data_s[data_s$RACE_ID == r, ]
    fit   <- clogit(reformulate(c(terms, "strata(RACE_ID)"), response = "IS_WINNER"), data = train)
    lp    <- as.matrix(test[terms]) %*% coef(fit)
    p     <- exp(lp) / sum(exp(lp))
    -log(p[test$IS_WINNER == 1])
  })
}

# engine log-loss (logged WIN_PROB of the actual winner) and uniform baseline
winners <- data |> filter(IS_WINNER == 1)
cv_results <- data.frame(
  RACE_ID      = races,
  uniform      = log(race_level$field_size[match(races, race_level$RACE_ID)]),
  engine       = -log(winners$WIN_PROB[match(races, winners$RACE_ID)]),
  market_only  = cv_fit("C_MARKET"),
  market_class = cv_fit(c("C_MARKET", "C_CLASS")),
  full         = cv_fit(core_components)
)

# mean log-loss per model (lower = better)
colMeans(cv_results[-1])
```

```
##      uniform       engine  market_only market_class         full
##     1.999653     1.668264     1.752013     1.770443     2.137574
```

```r
# run test
t.test(cv_results$engine, cv_results$full, paired = TRUE)
```

```
##
##  Paired t-test
##
## data:  cv_results$engine and cv_results$full
## t = -2.4008, df = 24, p-value = 0.02447
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.87276059 -0.06585879
## sample estimates:
## mean difference
##      -0.4693097
```

```r
decide(t.test(cv_results$engine, cv_results$full, paired = TRUE)$p.value)
```

```
## p-value = 0.02447 -> reject H0
```

```r
# -- mean log-loss: engine 1.67, market only 1.75, market + class 1.77, uniform 2.00, full 9-component
# -- engine vs full: mean difference = -0.47 (95% CI -0.87 to -0.07), p-value = 0.02447
# -- reject H0
# -- out of sample, the engine's probabilities beat the fitted 9-component model; the full model scored


# ** OUT-OF-SAMPLE: ENGINE VS MARKET ONLY ** ------------------------------------------------------
# H0: the engine and the market-only model have the same mean log-loss on held-out races
# H1: the mean log-loss differs between the engine and the market-only model
# alpha = 0.05
# test: paired t-test on per-race log-loss

# run test
t.test(cv_results$engine, cv_results$market_only, paired = TRUE)
```

```
##
##  Paired t-test
##
## data:  cv_results$engine and cv_results$market_only
## t = -0.65287, df = 24, p-value = 0.52
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.3485003  0.1810032
## sample estimates:
## mean difference
##     -0.08374854
```

```r
# -- mean difference (engine minus market only) = -0.084 (95% CI -0.35 to 0.18), p-value = 0.52
# -- fail to reject H0
# -- the engine's log-loss was not distinguishable from the market-only model's; the engine looks sligh


# ** AGAINST-THE-MARKET PICKS: MODEL TOP 3, MARKET NOT ** ------------------------------------------
# H0: horses the model ranks top 3 but the market ranks outside its top 3 finish in the top 3 at the ch
# H1: these horses finish in the top 3 more often than chance
# alpha = 0.05
# test: one-sided exact binomial

# find sneaky picks (market rank 1 = lowest odds)
sneaky <- data |>
  group_by(RACE_ID) |>
  mutate(mkt_rank = rank(ODDS, ties.method = "min")) |>
  ungroup() |>
  filter(RANK <= 3, mkt_rank > 3)

nrow(sneaky)
```

```
## [1] 20
```

```r
n_distinct(sneaky$RACE_ID)
```

```
## [1] 16
```

```r
sum(sneaky$IN_TOP3)
```

```
## [1] 8
```

```r
mean(3 / sneaky$FIELD_SIZE)
```

```
## [1] 0.3771429
```

```r
# run test
binom.test(sum(sneaky$IN_TOP3), nrow(sneaky),
           p = mean(3 / sneaky$FIELD_SIZE), alternative = "greater")
```

```
##
##  Exact binomial test
##
## data:  sum(sneaky$IN_TOP3) and nrow(sneaky)
## number of successes = 8, number of trials = 20, p-value = 0.5001
## alternative hypothesis: true probability of success is greater than 0.3771429
## 95 percent confidence interval:
##  0.2170686 1.0000000
## sample estimates:
## probability of success
##                    0.4
```

```r
# -- 8 of 20 sneaky picks finished top 3 (40%) vs 37.7% chance; p-value = 0.5001
# -- fail to reject H0
# -- the model's picks that the market ranks outside its top 3 finished top 3 at about the chance rate,
```